1. Student Details

Field	Details
Name	Ena Banga
AICTE ID Number	STUff9ef1cf3e3661788781209
Email	madhumitabanga@gmail.com
About Me	I am a final-year Electrical Engineering student at IEM KOLKATA and a Diploma in Data Science student at IIT Madras. Passionate about data science, artificial intelligence, and core electrical domain. I also love painting, dancing, karate, creative writing, and exploring new technologies. I have also a Diploma in Programming Degree from IIT Madras.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

To design and build a role-based, single-page web application that lets a trekking organisation manage treks end-to end: admins create and oversee treks and staff, staff manage the treks assigned to them, and users can search, browse and book open treks, receive notifications, and export their booking history.

Approach:

The application was built as a Flask REST API backend paired with a single-page Vue 3 frontend (loaded via CDN, no build step) styled with Bootstrap 5. All persistence uses raw SQLite (sqlite3) with foreign-key constraints and CHECK constraints enforced at the schema level rather than through an ORM. Authentication is session-based, with role checks (admin / staff / user) enforced through reusable route decorators. Time-consuming or repeating operations — booking-history CSV export, trek-reminder notifications, and a monthly admin summary — are offloaded to Celery workers backed by Redis, with the app automatically falling back to a synchronous, in-memory cache implementation when Redis is not available, so the app keeps working in a minimal environment.

3. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend framework; serves the REST API and the single HTML shell page
Vue 3 (CDN build)	Reactive single-page frontend; no build tooling required
Bootstrap 5	Frontend styling and responsive layout
Chart.js	Client-side charts for dashboard/analytics views
SQLite3 (raw)	Lightweight relational database, accessed directly via the sqlite3 module
Technology / Library	**Purpose**
Werkzeug security	Password hashing (generate_password_hash / check_password_hash) and session handling
Celery	Background task queue for async CSV export, reminders, and scheduled admin reports

Redis	Celery message broker/result backend, and cache store for the open-treks listing

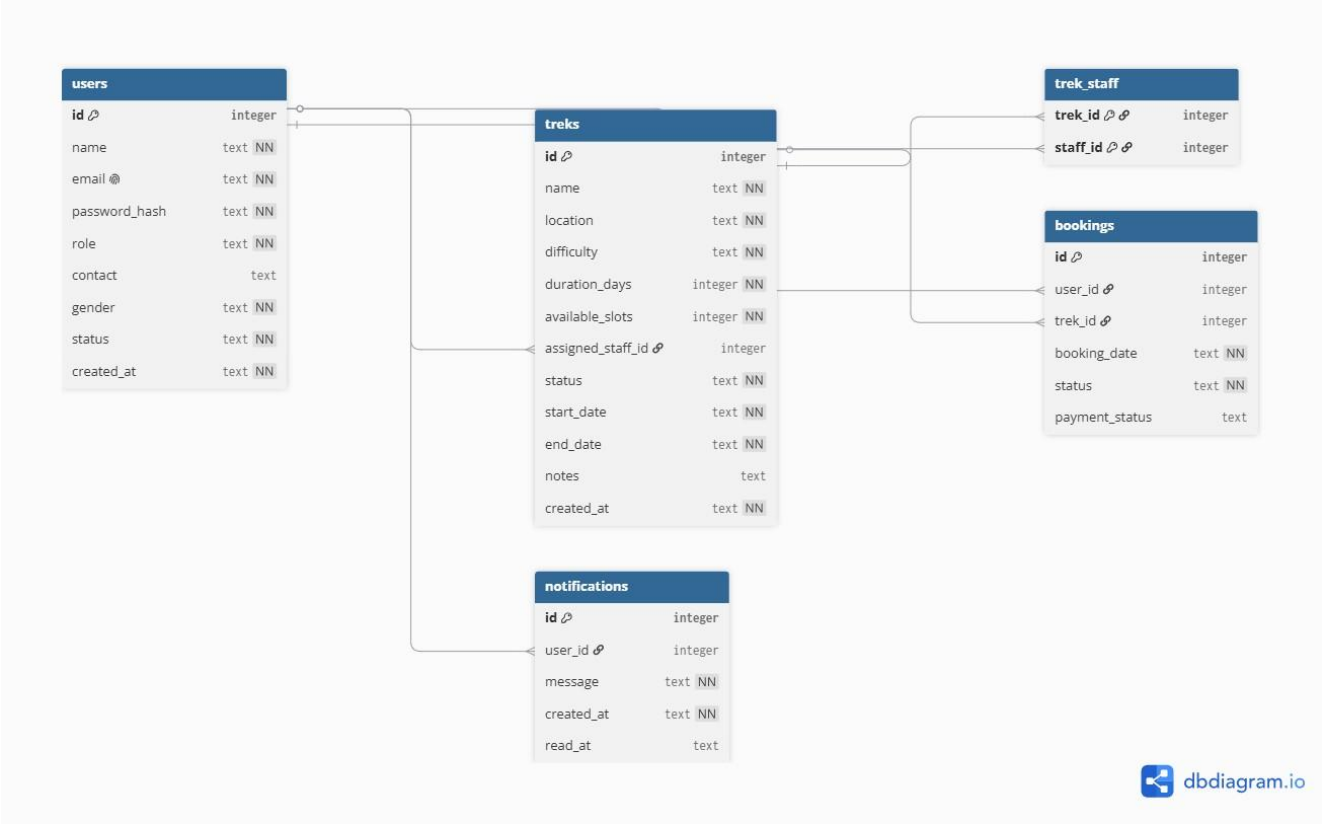
4. Database Schema / ER Diagram

Tables:

- users — id, name, email (unique), password_hash, role (admin/staff/user), contact, gender, status (active/inactive/blacklisted), created_at
- treks — id, name, location, difficulty (Easy/Moderate/Hard), duration_days, available_slots, assigned_staff_id (FK → users), status (Pending/Approved/Open/Closed/Completed), start_date, end_date, notes, created_at
- trek_staff — trek_id, staff_id (composite primary key); join table for the many-to-many assignment of staff to treks
- bookings — id, user_id (FK), trek_id (FK), booking_date, status (Booked/Cancelled/Completed), payment_status, UNIQUE(user_id, trek_id)
- notifications — id, user_id (FK, nullable), message, created_at, read_at

Relationships:

- users (staff) → treks : one-to-many (assigned_staff_id)
- users (staff) ↔ treks : many-to-many, via trek_staff
- users → bookings : one-to-many; treks → bookings : one-to-many
- users → notifications : one-to-many **ER Diagram:**



5. API Resource Endpoints

Endpoint	Method	Description
/api/register	POST	Register a new user or staff account
/api/login	POST	Authenticate and start a session
/api/logout	POST	Clear the current session
/api/me	GET	Return the currently logged-in user
/api/profile	PUT	Update the logged-in user's profile
/api/treks	GET	List treks, with search/filter support (q, difficulty, location, duration)
/api/treks	POST	Create a new trek (admin only)
/api/treks/<id>	PUT	Update a trek (admin, or staff assigned to it)
/api/treks/<id>	DELETE	Remove a trek and its bookings (admin only)
/api/staff	POST	Create a new staff account (admin only)
/api/users	GET	List/search users (admin only)
/api/users/<id>/status	PUT	Activate, deactivate, or blacklist a user (admin only)
/api/bookings	POST	Book a trek (user only), with slot and duplicate-booking checks
/api/bookings	GET	List bookings, scoped to the caller's role
/api/bookings/<id>	PUT	Update a booking's status
/api/dashboard	GET	Role-specific dashboard summary (admin stats, staff's assigned treks, or user's available/booked treks)
/api/exports/bookings	POST	Export the caller's booking history as CSV (async via Celery, with a synchronous fallback)

6. Architecture and Features

Architecture Overview:

- app.py — single-file Flask application: route definitions, Celery task definitions, and DB helpers
- templates/index.html — minimal HTML shell that mounts the Vue app on a single #app div
- static/app.js — Vue 3 application logic: views, state, and calls to the REST API
- static/styles.css — custom styling layered on top of Bootstrap
- tma.sqlite3 — SQLite database file, initialised and seeded on first run via init_db()

Implemented Features:

- Registration, login, logout with hashed passwords and session-based authentication
- Role-based access control for admin, staff, and user roles via reusable decorators
- Trek search/filter by keyword, difficulty, location, and duration
- Many-to-many staff assignment to treks
- Booking flow with available-slot checks and duplicate-booking prevention

- Notifications generated on booking, export completion, and scheduled reminders
- Role-specific analytics dashboard, visualised with Chart.js

Additional / Advanced Features:

- Asynchronous booking-history CSV export via Celery, with automatic synchronous fallback when Redis/Celery is unavailable
- Scheduled background jobs (Celery beat): daily trek-reminder notifications and a monthly admin summary report
- Short-lived caching (60s) of the open-treks listing, backed by Redis or an in-memory fallback cache
- Safety check surfacing a warning when a female user books a trek with no female staff assigned